

UNIT V SEARCHING, SORTING AND HASHING TECHNIQUES

Searching- Linear Search – Binary Search. Sorting – Bubble sort – Selection sort – Insertion sort
Hashing- Hash Functions – Separate Chaining – Open Addressing – Rehashing

What is Search?

Search is a process of **finding a value in a list of values**. In other words, searching is the **process of locating given value position in a list of values**.

Linear Search Algorithm (Sequential Search Algorithm)

Linear search algorithm finds a given element in a list of elements with $O(n)$ **time complexity** where **n** is total number of elements in the list. This search process starts comparing search element with the first element in the list. If both are matched then result is element found otherwise search element is compared with the next element in the list. Repeat the same until search element is compared with the last element in the list, if that last element also doesn't match, then the result is "Element not found in the list". That means, the search element is compared with element by element in the list.

Linear search is implemented using following steps...

- **Step 1** - Read the search element from the user.
- **Step 2** - Compare the search element with the first element in the list.
- **Step 3** - If both are matched, then display "Given element is found!!!" and terminate the function
- **Step 4** - If both are not matched, then compare search element with the next element in the list.
- **Step 5** - Repeat steps 3 and 4 until search element is compared with last element in the list.
- **Step 6** - If last element in the list also doesn't match, then display "Element is not found!!!" and terminate the function.

Example

Consider the following list of elements and the element to be searched...

list 0 1 2 3 4 5 6 7
 [65][20][10][55][32][12][50][99]

search element 12

Step 1:

search element (12) is compared with first element (65)

list 0 1 2 3 4 5 6 7
 [65][20][10][55][32][12][50][99]

12

Both are not matching. So move to next element

Step 2:

search element (12) is compared with next element (20)

list 0 1 2 3 4 5 6 7
 [65][20][10][55][32][12][50][99]

12

Both are not matching. So move to next element

Step 3:

search element (12) is compared with next element (10)

list 0 1 2 3 4 5 6 7
 [65][20][10][55][32][12][50][99]

12

Both are not matching. So move to next element

Step 4:

search element (12) is compared with next element (55)

list 0 1 2 3 4 5 6 7
 [65][20][10][55][32][12][50][99]

12

Both are not matching. So move to next element

Step 5:

search element (12) is compared with next element (32)

list 0 1 2 3 4 5 6 7
 [65][20][10][55][32][12][50][99]

12

Both are not matching. So move to next element

Step 6:

search element (12) is compared with next element (12)

list 0 1 2 3 4 5 6 7
 [65][20][10][55][32][12][50][99]

12

Both are matching. So we stop comparing and display element found at index 5.

IMPLEMENTATION OF THE LINEAR SEARCH USING C

```
#include<stdio.h>
#include<conio.h>
void main(){
    int list[20],size,i,sElement;
    printf("Enter size of the list: ");
    scanf("%d",&size);
    printf("Enter any %d integer values: ",size);
    for(i = 0; i < size; i++)
        scanf("%d",&list[i]);
    printf("Enter the element to be Search: ");
    scanf("%d",&sElement);
    // Linear Search Logic
    for(i = 0; i < size; i++)
    {
        if(sElement == list[i])
        {
            printf("Element is found at %d index", i);
            break;
        }
    }
    if(i == size)
        printf("Given element is not found in the list!!!");
    getch();
}
```

Binary Search Algorithm

Binary search algorithm finds a given element in a list of elements with $O(\log n)$ time complexity where n is total number of elements in the list. The binary search algorithm can be used with only a sorted list of elements. That means the binary search is used only with a list of elements that are already arranged in an order. The binary search can not be used for a list of elements arranged in random order. This search process starts comparing the search element with the middle element in the list. If both are matched, then the result is "element found". Otherwise, we check whether the search element is smaller or larger than the middle element in the list. If the search element is smaller, then we repeat the same process for the left sublist of the middle element. If the search element is larger, then we repeat the same process for the right sublist of the middle element. We repeat this process until we find the search element in the list or until we left with a sublist of only one element. And if that element also doesn't match with the search element, then the result is "Element not found in the list".

Binary search is implemented using following steps...

- **Step 1** - Read the search element from the user.
- **Step 2** - Find the middle element in the sorted list.
- **Step 3** - Compare the search element with the middle element in the sorted list.
- **Step 4** - If both are matched, then display "Given element is found!!!" and terminate the function.
- **Step 5** - If both are not matched, then check whether the search element is smaller or larger than the middle element.
- **Step 6** - If the search element is smaller than middle element, repeat steps 2, 3, 4 and 5 for the left sublist of the middle element.
- **Step 7** - If the search element is larger than middle element, repeat steps 2, 3, 4 and 5 for the right sublist of the middle element.
- **Step 8** - Repeat the same process until we find the search element in the list or until sublist contains only one element.
- **Step 9** - **If that element also doesn't match with the search element, then display "Element is not found in the list!!!" and terminate the function.**

Example

Consider the following list of elements and the element to be searched...

list

0	1	2	3	4	5	6	7	8
10	12	20	32	50	55	65	80	99

search element **12**

Step 1:

search element (12) is compared with middle element (50)

list

0	1	2	3	4	5	6	7	8
10	12	20	32	50	55	65	80	99

12

Both are not matching. And 12 is smaller than 50. So we search only in the left sublist (i.e. 10, 12, 20 & 32).

list

0	1	2	3	4	5	6	7	8
10	12	20	32	50	55	65	80	99

Step 2:

search element (12) is compared with middle element (12)

list

0	1	2	3	4	5	6	7	8
10	12	20	32	50	55	65	80	99

12

Both are matching. So the result is "Element found at index 1"

search element **80**

Step 1:

search element (80) is compared with middle element (50)

list

0	1	2	3	4	5	6	7	8
10	12	20	32	50	55	65	80	99

80

Both are not matching. And 80 is larger than 50. So we search only in the right sublist (i.e. 55, 65, 80 & 99).

list

0	1	2	3	4	5	6	7	8
10	12	20	32	50	55	65	80	99

Step 2:

search element (80) is compared with middle element (65)

list

0	1	2	3	4	5	6	7	8
10	12	20	32	50	55	65	80	99

80

Both are not matching. And 80 is larger than 65. So we search only in the right sublist (i.e. 80 & 99).

list

0	1	2	3	4	5	6	7	8
10	12	20	32	50	55	65	80	99

Step 3:

search element (80) is compared with middle element (80)

list

0	1	2	3	4	5	6	7	8
10	12	20	32	50	55	65	80	99

80

Both are not matching. So the result is "Element found at index 7"

IMPLEMENTATION OF BINARY SEARCH USING C:

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int first, last, middle, size, i, sElement, list[100];
    clrscr();
    printf("Enter the size of the list: ");
    scanf("%d",&size);
    printf("Enter %d integer values in Assending order\n", size);
    for (i = 0; i < size; i++)
        scanf("%d",&list[i]);
    printf("Enter value to be search: ");
    scanf("%d", &sElement);
    first = 0;
    last = size - 1;
    middle = (first+last)/2;
    while (first <= last) {
        if (list[middle] < sElement)
            first = middle + 1;
        else if (list[middle] == sElement) {
            printf("Element found at index %d.\n",middle);
            break;
        }
        else
            last = middle - 1;
        middle = (first + last)/2;
    }
    if (first > last)
        printf("Element Not found in the list.");
}
```

```
    getch();  
}
```

SORTING:

Sorting refers to arranging data in a particular format. Sorting algorithm specifies the way to arrange data in a particular order.

The importance of sorting lies in the fact that data searching can be optimized to a very high level, if data is stored in a sorted manner.

Following are some of the examples of sorting in real-life scenarios –

- **Telephone Directory** – The telephone directory stores the telephone numbers of people sorted by their names, so that the names can be searched easily.
- **Dictionary** – The dictionary stores words in an alphabetical order so that searching of any word becomes easy.

There are several types of sorting in data structures namely – **bubble sort, insertion sort, selection sort, bucket sort, heap sort, quick sort, radix sort** etc.

BUBBLE SORT:

Bubble sort is a simple sorting algorithm. This sorting algorithm is comparison-based algorithm in which each pair of adjacent elements is compared and the elements are swapped if they are not in order. This algorithm is not suitable for large data sets as its average and worst case complexity are of $O(n^2)$ where **n** is the number of items.

How Bubble Sort Works?

We take an unsorted array for our example. Bubble sort takes $O(n^2)$ time so we're keeping it short and precise.



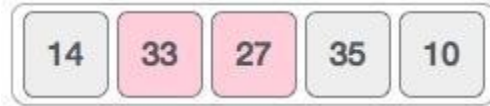
Bubble sort starts with very first two elements, comparing them to check which one is greater.



In this case, value 33 is greater than 14, so it is already in sorted locations. Next, we compare 33 with 27.



We find that 27 is smaller than 33 and these two values must be swapped.



The new array should look like this –



Next we compare 33 and 35. We find that both are in already sorted positions.



Then we move to the next two values, 35 and 10.



We know then that 10 is smaller 35. Hence they are not sorted.



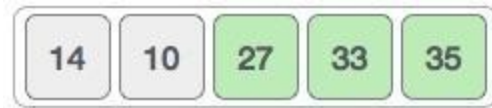
We swap these values. We find that we have reached the end of the array. After one iteration, the array should look like this –



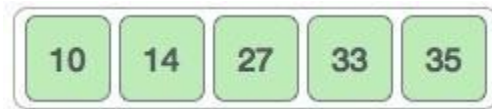
To be precise, we are now showing how an array should look like after each iteration. After the second iteration, it should look like this –



Notice that after each iteration, at least one value moves at the end.



And when there's no swap required, bubble sort learns that an array is completely sorted.



Now we should look into some practical aspects of bubble sort.

ALGORITHM:

```
begin BubbleSort(list)

  for all elements of list
    if list[i] > list[i+1]
      swap(list[i], list[i+1])
    end if
  end for

  return list

end BubbleSort
```

IMPLEMENTATION USING C :

```
#include <stdio.h>
#include <stdbool.h>

#define MAX 10

int list[MAX] = {1,8,4,6,0,3,5,2,7,9};

void display() {
  int i;
  printf("[");

  // navigate through all items
  for(i = 0; i < MAX; i++) {
    printf("%d ", list[i]);
  }

  printf("]\n");
}
```

```

}

void bubbleSort() {
    int temp;
    int i,j;

    bool swapped = false;

    // loop through all numbers
    for(i = 0; i < MAX-1; i++) {
        swapped = false;

        // loop through numbers falling ahead
        for(j = 0; j < MAX-1-i; j++) {
            printf("    Items compared: [ %d, %d ] ", list[j],list[j+1]);

            // check if next number is lesser than current no
            // swap the numbers.
            // (Bubble up the highest number)

            if(list[j] > list[j+1]) {
                temp = list[j];
                list[j] = list[j+1];
                list[j+1] = temp;

                swapped = true;
                printf("=> swapped [%d, %d]\n",list[j],list[j+1]);
            } else {
                printf("=> not swapped\n");
            }
        }

        // if no number was swapped that means
        // array is sorted now, break the loop.
        if(!swapped) {
            break;
        }

        printf("Iteration %d#: ",(i+1));
        display();
    }
}

void main() {
    printf("Input Array: ");

```

```

display();
printf("\n");

bubbleSort();
printf("\nOutput Array: ");
display();
}

```

SELECTION SORT:

Selection sort is a simple sorting algorithm. This sorting algorithm is an in-place comparison-based algorithm in which the list is divided into two parts, the sorted part at the left end and the unsorted part at the right end. Initially, the sorted part is empty and the unsorted part is the entire list.

The smallest element is selected from the unsorted array and swapped with the leftmost element, and that element becomes a part of the sorted array. This process continues moving unsorted array boundary by one element to the right.

This algorithm is not suitable for large data sets as its average and worst case complexities are of $O(n^2)$, where n is the number of items.

How Selection Sort Works?

Consider the following depicted array as an example.



For the first position in the sorted list, the whole list is scanned sequentially. The first position where 14 is stored presently, we search the whole list and find that 10 is the lowest value.



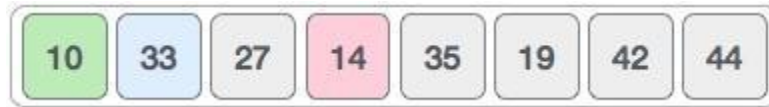
So we replace 14 with 10. After one iteration 10, which happens to be the minimum value in the list, appears in the first position of the sorted list.



For the second position, where 33 is residing, we start scanning the rest of the list in a linear manner.



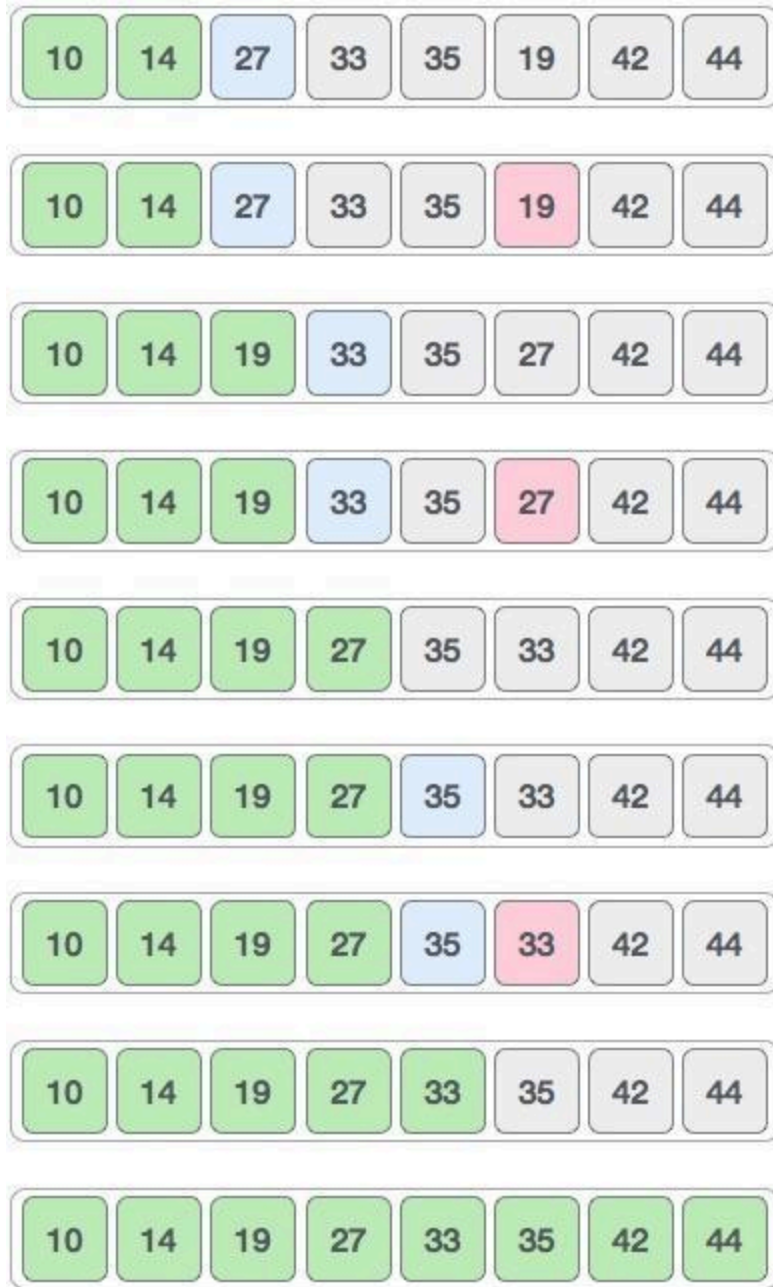
We find that 14 is the second lowest value in the list and it should appear at the second place. We swap these values.



After two iterations, two least values are positioned at the beginning in a sorted manner.



The same process is applied to the rest of the items in the array.



Now, let us learn some programming aspects of selection sort.

Algorithm

Step 1 – Set MIN to location 0

Step 2 – Search the minimum element in the list

Step 3 – Swap with value at location MIN

Step 4 – Increment MIN to point to next element

Step 5 – Repeat until list is sorted

IMPLEMENTATION IN C:

```

#include <stdio.h>
#include <stdbool.h>
#define MAX 7
intArray[MAX] = {4,6,3,2,1,9,7};
void printline(int count) {
    int i;
    for(i = 0;i < count-1;i++) {
        printf("=");
    }
    printf("\n");
}

void display() {
    int i;
    printf("[");
    // navigate through all items
    for(i = 0;i < MAX;i++) {
        printf("%d ", intArray[i]);
    }

    printf("]\n");
}

void selectionSort() {
    int indexMin,i,j;

    // loop through all numbers
    for(i = 0; i < MAX-1; i++) {

```

```

        // set current element as minimum
indexMin = i;

// check the element to be minimum
for(j = i+1; j < MAX; j++) {
    if(intArray[j] < intArray[indexMin]) {
        indexMin = j;
    }
}

if(indexMin != i) {
    printf("Items swapped: [ %d, %d ]\n" , intArray[i], intArray[indexMin]);

    // swap the numbers
    int temp = intArray[indexMin];
    intArray[indexMin] = intArray[i];
    intArray[i] = temp;
}

printf("Iteration %d#:",(i+1));
display();
}
}

void main() {
    printf("Input Array: ");
    display();
    prntline(50);
}

```

```
selectionSort();  
printf("Output Array: ");  
display();  
println(50);  
}
```

INSERTION SORT:

This is an in-place comparison-based sorting algorithm. Here, a sub-list is maintained which is always sorted.

An element which is to be 'insert'ed in this sorted sub-list, has to find its appropriate place and then it has to be inserted there. Hence the name, **insertion sort**.

The array is searched sequentially and unsorted items are moved and inserted into the sorted sub-list (in the same array). This algorithm is not suitable for large data sets as its average and worst case complexity are of $O(n^2)$, where **n** is the number of items.

How Insertion Sort Works?

We take an unsorted array for our example.



Insertion sort compares the first two elements.



It finds that both 14 and 33 are already in ascending order. For now, 14 is in sorted sub-list.



Insertion sort moves ahead and compares 33 with 27.



And finds that 33 is not in the correct position.



It swaps 33 with 27. It also checks with all the elements of sorted sub-list. Here we see that the sorted sub-list has only one element 14, and 27 is greater than 14. Hence, the sorted sub-list remains sorted after swapping.



By now we have 14 and 27 in the sorted sub-list. Next, it compares 33 with 10.



These values are not in a sorted order.



So we swap them.



However, swapping makes 27 and 10 unsorted.



Hence, we swap them too.



Again we find 14 and 10 in an unsorted order.



We swap them again. By the end of third iteration, we have a sorted sub-list of 4 items.



This process goes on until all the unsorted values are covered in a sorted sub-list. Now we shall see some programming aspects of insertion sort.

Algorithm

Now we have a bigger picture of how this sorting technique works, so we can derive simple steps by which we can achieve insertion sort.

- Step 1** – If it is the first element, it is already sorted. return 1;
- Step 2** – Pick next element
- Step 3** – Compare with all elements in the sorted sub-list
- Step 4** – Shift all the elements in the sorted sub-list that is greater than the value to be sorted
- Step 5** – Insert the value
- Step 6** – Repeat until list is sorted

IMPLEMENTATION IN C

```
#include <stdio.h>

#include <stdbool.h>

#define MAX 7

int intArray[MAX] = {4,6,3,2,1,9,7};

void printline(int count) {
    int i;
    for(i = 0; i < count-1; i++) {
        printf("=");
    }
    printf("\n");
}

void display() {
    int i;
    printf("[");
    // navigate through all items
```

```

for(i = 0; i < MAX; i++) {
    printf("%d ", intArray[i]);
}
printf("\n");
}

void insertionSort() {
    int valueToInsert;
    int holePosition;
    int i;

    // loop through all numbers
    for(i = 1; i < MAX; i++) {
        // select a value to be inserted.
        valueToInsert = intArray[i];
        // select the hole position where number is to be inserted
        holePosition = i;

        // check if previous no. is larger than value to be inserted
        while (holePosition > 0 && intArray[holePosition-1] > valueToInsert) {
            intArray[holePosition] = intArray[holePosition-1];
            holePosition--;
            printf(" item moved : %d\n" , intArray[holePosition]);
        }
        if(holePosition != i) {
            printf(" item inserted : %d, at position : %d\n" , valueToInsert, holePosition);
            // insert the number at hole position
            intArray[holePosition] = valueToInsert;
        }
        printf("Iteration %d#:", i);
        display();
    }
}

```

```

    }
}

void main() {
    printf("Input Array: ");
    display();
    printline(50);
    insertionSort();
    printf("Output Array: ");
    display();
    printline(50);
}

```

Quick Sort

- Quicksort is a divide and conquer algorithm. The basic idea is to find a “pivot” item in the array and compare all other items with pivot element.
- Shift items such that all of the items before the pivot are less than the pivot value and all the items after the pivot are greater than the pivot value.
- After that, recursively perform the same operation on the items before and after the pivot. Find a “pivot” item in the array.
- This item is the basis for comparison for a single round.

Start a pointer (the left pointer) at the first item in the array.

Start a pointer (the right pointer) at the last item in the array.

1. Assume $A[0]$ =pivot which is the left. i.e pivot=left.
2. Set i =left+1; i.e $A[1]$;
3. Set j =right. ie. $A[6]$ if there are 7 elements in the array

4. If $A[\text{pivot}] > A[i]$, increment i and if $A[j] > A[\text{pivot}]$, then decrement j , Otherwise swap $A[i]$ and $A[j]$ element.
5. If $i = j$, then swap $A[\text{pivot}]$ and $A[j]$.

Consider the following unsorted list of elements...



Define pivot, left & right. Set pivot = 0, left = 1 & right = 7. Here '7' indicates 'size-1'.



Compare List[left] with List[pivot]. If List[left] is greater than List[pivot] then stop left otherwise move left to the next.

Compare List[right] with List[pivot]. If List[right] is smaller than List[pivot] then stop right otherwise move right to the previous.

Repeat the same until $\text{left} \geq \text{right}$.

If both left & right are stopped but $\text{left} < \text{right}$ then swap List[left] with List[right] and continue the process.

If $\text{left} \geq \text{right}$ then swap List[pivot] with List[right].



Compare List[left] < List[pivot] as it is true increment left by one and repeat the same, left will stop at 8.

Compare List[right] > List[pivot] as it is true decrement right by one and repeat the same, right will stop at 2.



Here left & right both are stopped and left is not greater than right so we need to swap List[left] and List[right]



Compare List[left] < List[pivot] as it is true increment left by one and repeat the same, left will stop at 6.

Compare List[right] > List[pivot] as it is true decrement right by one and repeat the same, right will stop at 4.



Here left & right both are stopped and left is greater than right so we need to swap List[pivot] and List[right]



Here we can observe that all the numbers to the left side of 5 are smaller and right side are greater. That means 5 is placed in its correct position.

Repeat the same process on the left sublist and right sublist to the number 5.



In the left sublist as there are no smaller number than the pivot left will keep on moving to the next and stops at last number. As the List[right] is smaller, right stops at same position. Now left and right both are equal so we swap pivot with right.



In the right sublist left is greater than the pivot, left will stop at same position.

As the List[right] is greater than List[pivot], right moves towards left and stops at pivot number position.

Now left > right so we swap pivot with right. (6 is swap by itself).



Repeat the same recursively on both left and right sublists until all the numbers are sorted.

The final sorted list will be as follows...



Quick Sort routine:

```
void Quicksort ( int a [ ], int left, int right )
```

```
{
```

```
int i, j, p, temp;
```

```
if ( left < right )
```

```
{
```

```
p = left;
```

```
i = left + 1;
```

```
j = right;
```

```
while ( i < j )
```

```
{
```

```
while ( a [ i ] <= a [ p ] )
```

```
i = i + 1;
```

```
while ( a [ j ] > a [ p ] )
```

```
j = j - 1;
```

```
if ( i < j )
```

```
{
```

```
}
```

```
temp = a [ i ];
```

```
a [ i ] = a [ j ];
```

```
a [ j ] = temp;
```

```
}
```

```
temp = a [ p ];
```

```
a [ p ] = a [ j ];
```

```
a [ j ] = temp;
```

```
quicksort ( a, left, j - 1 );
quicksort ( a, j + 1, right );
} }
```

Program for Quick sort

```
#include<stdio.h>

void quicksort (int [10], int, int ) ;

void main( )
{
int a[20], n, i ;
printf("Enter size of the array:" );
scanf("%d",&n);
printf( " Enter the numbers :");
for ( i = 0 ; i < n ; i ++ )
scanf ("%d",&a[i]);
quicksort ( a , 0 , n - 1 );
printf ( " Sorted elements: " );
for ( i = 0 ; i < n ; i ++ )
printf ("%d\t",a[ i]);
}

void quicksort ( int a[10], int left, int right )
{
int p, j, temp, i ;
if ( left < right )
{
p = left ;
i = left ;
j = right ;
while ( i < j )
```

```

{
}
while(a[i]<= a[p] && i<right )
i++ ;
while ( a [ j ] > a [ p ] )
j--;
if ( i < j )
{
}
temp = a [ i ] ;
a [ i ] = a [ j ] ;
a[ j ] = temp ;
temp = a [ p ] ;
a [ p ] = a [ j ] ;
a [ j ] =temp ;
quicksort ( a , left , j - 1 ) ;
quicksort ( a , j + 1 , right ) ;
}
}

```

OUTPUT:

Enter size of the array:8

Enter the numbers :40 20 70 14 60 61 97 30

Sorted elements: 14 20

30 40 60 61 70 97

Advantages of Quick sort

- Fast and efficient as it deals well with a huge list of items.
- No additional storage is required.

Disadvantages of Quick sort

- The difficulty of implementing the partitioning algorithm.

Merge Sort

- Merge sort is a sorting algorithm that uses the divide, conquer, and combine algorithmic paradigm.
- Divide means partitioning the n -element array to be sorted into two sub-arrays of $n/2$ elements.
- If there are more elements in the array, divide A into two sub-arrays, A_1 and A_2 , each containing about half of the elements of A .
- Conquer means sorting the two sub-arrays recursively using merge sort.
- Combine means merging the two sorted sub-arrays of size $n/2$ to produce the sorted array of n elements.

The basic steps of a merge sort algorithm are as follows:

If the array is of length 0 or 1, then it is already sorted.

Otherwise, divide the unsorted array into two sub-arrays of about half the size.

Use merge sort algorithm recursively to sort each sub-array.

Merge the two sub-arrays to form a single sorted list.

Merge Sort routine:

```
void Merge_sort (int a [ ], int temp [ ], int n )
{
    msort ( a , temp , 0 , n - 1 ) ;
}
```

```
void msort ( int a[ ], int temp [ ], int left , int right )
```

```
{
```

```
    int center ;
```

```
        if( left < right )
```

```
        {
```

```
            center = ( left + right ) / 2 ;
```

```
            msort ( a , left , center ) ;
```

```
            msort ( a , temp , center + 1 , right ) ;
```

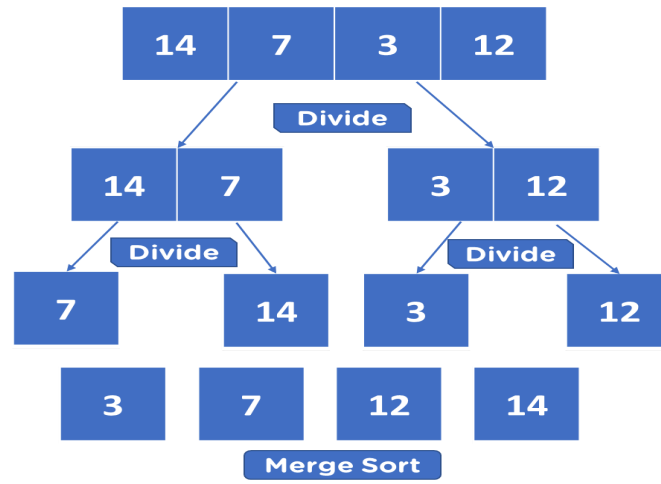
```
void merge ( int a [ ], int temp [ ], int n , int left , int center , int right )
```

```
{
```

```
    int i = 0 , j , left_end = center , center = center + 1 ;
```

```
    while( ( left <= left_end ) && ( center <= right ) )
```

{



```

        if( a [ left ] <= a [ center ] )
        {
            temp [ i ] = a [ left ] ;
            i ++ ;
            left ++ ;
        }
        else
        {
            temp [ i ] = a [ center ] ;
            i ++ ;
            center ++ ;
        }
    }
    while( left <= left_end )
    {
        temp [ i ] = a [ left ] ;
        left ++ ;
        i ++ ;
    }
    while( center <= right )
    {
        temp [ i ] = a [ center ] ;
        center ++ ;
        i ++ ;
    }
    for ( i = 0 ; i < n ; i ++ )
        print temp [ i ] ;
}

```

Program for merge sort

```

#include<stdio.h>
void mergesort(int a[],int i,int j);
void merge(int a[],int i1,int j1,int i2,int j2);
int main()
{
    int a[30],n,i;
    printf("Enter no of elements:");
    scanf("%d",&n);
    printf("Enter array
elements:"); for(i=0;i<n;i++)
    scanf("%d",&a[i]);
    mergesort(a,0,n-1);
    printf("\nSorted array is :");
}

```

```

    for(i=0;i<n;i++)
    printf("%d ",a[i]);
    return 0;
}

void mergesort(int a[],int i,int j)
{
    int mid; if(i<j)
    {
        mid=(i+j)/2;
        mergesort(a,i,mid);    //left recursion
        mergesort(a,mid+1,j);  //right recursion
        merge(a,i,mid,mid+1,j); //merging of two sorted sub-arrays
    }
}

void merge(int a[],int i1,int j1,int i2,int j2)
{
    int temp[50]; //array used for merging
    int i,j,k;
    i=i1; //beginning of the first list
    j=i2; //beginning of the second
    list k=0;
    while(i<=j1 && j<=j2) //while elements in both lists
    {
        if(a[i]<a[j])
            temp[k++]=a[i++];
        else
            temp[k++]=a[j++];
    }

    while(i<=j1) //copy remaining elements of the first list
        temp[k++]=a[i++];
    while(j<=j2) //copy remaining elements of the second list
        temp[k++]=a[j++]; //Transfer elements from temp[] back to a[]
    for(i=i1,j=0;i<=j2;i++,j++)
        a[i]=temp[j];
}

```

OUTPUT:

Enter no of elements:8

Enter array elements:24 13 26 1 2 27 38 15

Sorted array is :1 2 13 15 24 26 27 38

Advantages of Merge sort

- Mergesort is well-suited for sorting really huge amounts of data that does not fit into memory.
- It is fast and stable algorithm

Disadvantages of Merge sort

- Merge sort uses a lot of memory.
- It uses extra space proportional to number of element n.
- This can slow it down when attempting to sort very large data

HASH TABLE:

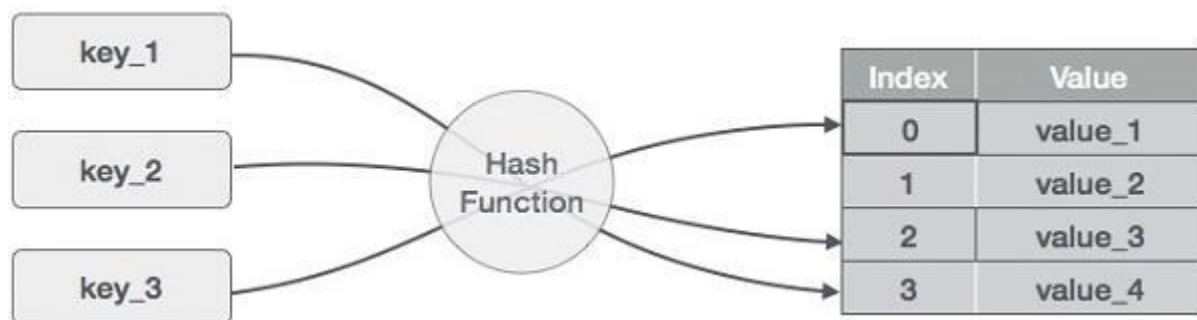
Hash Table is a data structure which stores data in an associative manner. In a hash table, data is stored in an array format, where each data value has its own unique index value.

Hash Table uses an array as a storage medium and uses hash technique to generate an index where an element is to be inserted or is to be located from.

HASHING:

Hashing is a technique to convert a range of key values into a range of indexes of an array.

Item are in the (key,value) format.



- (1,20) (2,70) (42,80)(4,25) (12,44) (14,32)
- (17,11) (13,78) (37,98)

Sr.No.	Key	Hash	Array Index
1	1	$1 \% 20 = 1$	1
2	2	$2 \% 20 = 2$	2
3	42	$42 \% 20 = 2$	2
4	4	$4 \% 20 = 4$	4
5	12	$12 \% 20 = 12$	12
6	14	$14 \% 20 = 14$	14
7	17	$17 \% 20 = 17$	17
8	13	$13 \% 20 = 13$	13
9	37	$37 \% 20 = 17$	17

Hash Collision

When the hash function generates the same index for multiple keys, there will be a conflict (what value to be stored in that index). This is called a **hash collision**.

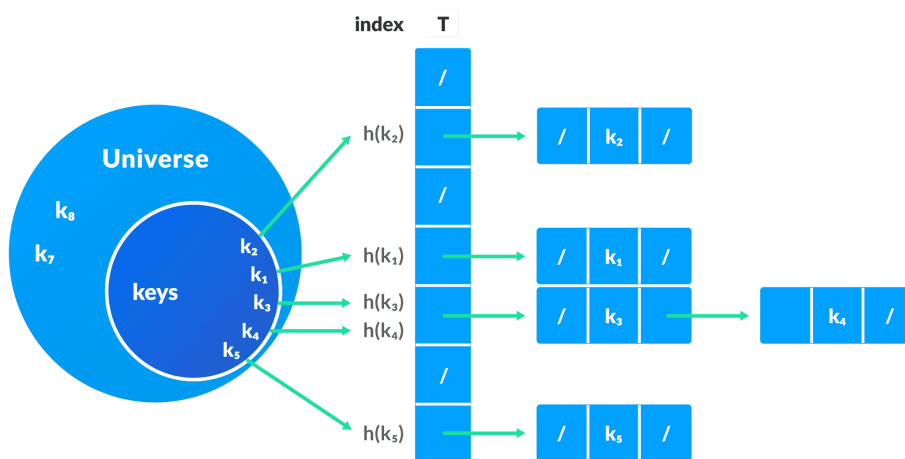
We can resolve the hash collision using one of the following techniques.

- Collision resolution by chaining
- Open Addressing: Linear/Quadratic Probing and Double Hashing

1. Collision resolution by chaining

In chaining, if a hash function produces the same index for multiple elements, these elements are stored in the same index by using a doubly-linked list.

If j is the slot for multiple elements, it contains a pointer to the head of the list of elements. If no element is present, j contains NIL.



Collision Resolution using chaining

2. Open Addressing

Unlike chaining, open addressing doesn't store multiple elements into the same slot. Here, each slot is either filled with a single key or left NIL.

Different techniques used in open addressing are:

i. Linear Probing

In linear probing, collision is resolved by checking the next slot.

$$h(k, i) = (h'(k) + i) \bmod m$$
 where

- $i = \{0, 1, \dots\}$
- $h'(k)$ is a new hash function

If a collision occurs at $h(k, 0)$, then $h(k, 1)$ is checked. In this way, the value of i is incremented linearly.

The hashing technique is used to create an already used index of the array. In such a case, we can search the next empty location in the array by looking into the next cell until we find an empty cell. This technique is called linear probing.

Sr.No.	Key	Hash	Array Index	After Linear Probing, Array Index
1	1	$1 \% 20 = 1$	1	1
2	2	$2 \% 20 = 2$	2	2

3	42	$42 \% 20 = 2$	2	3
4	4	$4 \% 20 = 4$	4	4
5	12	$12 \% 20 = 12$	12	12
6	14	$14 \% 20 = 14$	14	14
7	17	$17 \% 20 = 17$	17	17
8	13	$13 \% 20 = 13$	13	13
9	37	$37 \% 20 = 17$	17	18

The problem with linear probing is that a cluster of adjacent slots is filled. When inserting a new element, the entire cluster must be traversed. This adds to the time required to perform operations on the hash table.

ii. Quadratic Probing

It works similar to linear probing but the spacing between the slots is increased (greater than one) by using the following relation.

$$h(k, i) = (h'(k) + c_1i + c_2i^2) \bmod m$$

where,

- c_1 and c_2 are positive auxiliary constants,
- $i = \{0, 1, \dots\}$

iii. Double hashing

If a collision occurs after applying a hash function $h(k)$, then another hash function is calculated for finding the next slot.

$$h(k, i) = (h_1(k) + ih_2(k)) \bmod m$$

Good Hash Functions

A good hash function may not prevent the collisions completely however it can reduce the number of collisions.

Here, we will look into different methods to find a good hash function

1. Division Method

If k is a key and m is the size of the hash table, the hash function $h()$ is calculated as:
 $h(k) = k \bmod m$

2. Multiplication Method

$h(k) = \lfloor m(kA \bmod 1) \rfloor$
where,

- $kA \bmod 1$ gives the fractional part kA ,
- $\lfloor \rfloor$ gives the floor value
- A is any constant. The value of A lies between 0 and 1. But, an optimal choice will be $\approx (\sqrt{5}-1)/2$ suggested by Knuth.

3. Universal Hashing

In Universal hashing, the hash function is chosen at random independent of keys.

Applications of Hash Table

Hash tables are implemented where

- constant time lookup and insertion is required
- cryptographic applications
- indexing data is required